DISCRETE MATHEMATICS (PCCO201) • PROBLEM-BASED LEARNING

SUM OF SQUARES OF FIRST *n* NATURAL NUMBERS

Formal Analytical Derivation via Mathematical Induction

TARGET PROPOSITION • P(N)

$\forall\ n \in \mathbb{N}$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = S(n) = \frac{n(n+1)(2n+1)}{6}$$

Base Step: S(1) = 1 • **Inductive Hypothesis:** P(k) is True • **Inductive Step:** P(k + 1)

“Can one formula prove the answer for every n?”

NEXT → PATTERN DISCOVERY

PROJECT TEAM MEMBERS

Academic Year 2025-2026

01 Kayastha Durgesh Manoj

02 Zalte Prem Dattatray

03 Unde Pranav Suresh

04 Harde Vaidehi Suklal

STAGE 1: PROBLEM FORMULATION & OBJECTIVE

Problem Overview & Problem Statement

1. The Problem

TARGET SERIES SUMMATION

$$S(n) = 1^2 + 2^2 + 3^2 + \cdots + n^2$$

Find a closed-form formula for the sum of squares of the first **n** natural numbers without manual addition.

`Input Domain: ∀ n ∈ ℕ = {1, 2, 3, …}`

Sequence Pattern Discovery

DISCRETE TERMS (I^2)

$1^2 \rightarrow 2^2 \rightarrow 3^2 \rightarrow 4^2 \rightarrow \ldots \rightarrow n^2$

ACCUMULATED VALUES S(N)

1 → 5 → 14 → 30 → … → S(n)

Values grow polynomially ➔ Leads to cubic conjecture.

2. Project Objective

PROPOSED CLOSED FORM

$$S(n) = \frac{n(n+1)(2n+1)}{6}$$

Prove that this formula is true for every natural number **n** using Mathematical Induction.

IMPLICATION: $P(k) \Rightarrow P(k+1)$

Universal Validity

Proves statements for all infinite natural numbers without exceptions.

Systematic Methodology

Provides a rigorous, step-by-step framework for sequences and series.

Algorithmic Bridge

The proved closed form enables optimal O(1) computational implementation.

“How can we prove that this formula works for every n?”

NEXT STAGE: MATHEMATICAL INDUCTION ➔

STAGE 2: PROOF METHODOLOGY

Why Mathematical Induction?

The Domino Effect: Infinite Succession

"If P(k) is true, what must we prove next?"



1 Base Case

Anchor truth at n = 1.

2 Inductive Hypothesis

Assume statement true for n = k.

3 Inductive Step

Deduce validity for n = k + 1.

$$[\ P(1) \land (\forall k,\ P(k) \Rightarrow P(k+1))\ ] \Rightarrow \forall n \in \mathbb{N},\ P(n) \text{ is True}$$

PRINCIPLE OF MATHEMATICAL INDUCTION

ANALYTICAL RIGOR • DISCRETE PROOF

Proof of the Sum-of-Squares Formula

1. BASE CASE (N = 1)

Verified ✓

LHS = $1^2 = 1$

RHS = $1(1 + 1)(2(1) + 1) / 6 = 6 / 6 = 1$

2. INDUCTIVE HYPOTHESIS

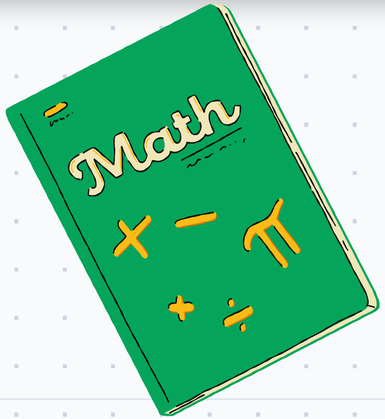
Assumed for k

Assume true for n = k:

$$1^2 + 2^2 + \cdots + k^2 = k(k + 1)(2k + 1) / 6$$

TARGET FORMULA FOR (k + 1)

$$S(k+1) = (k + 1)(k + 2)(2k + 3) / 6$$



3. INDUCTIVE STEP (ALGEBRAIC DERIVATION)

Add $(k+1)^2$

$$S(k+1) = [\ 1^2 + 2^2 + \cdots + k^2\ ] + (k+1)^2$$

$$= [\ k(k+1)(2k+1) / 6\ ] + (k+1)^2$$

$$= (k+1) \cdot [\ k(2k+1) + 6(k+1)\ ] / 6$$

$$= (k+1) \cdot [\ 2k^2 + 7k + 6\ ] / 6$$

$$= (k+1)(k+2)(2k+3) / 6$$ ✓ Q.E.D.

Hence, by Mathematical Induction, the formula is rigorously true for all n ≥ 1.

PBL Link: The inductive step validates recursion and invariant execution in computer science.

STAGE 3: ALGORITHMIC TRANSLATION

From Mathematical Proof to Algorithm

DISCRETE MATH
Proven Formula
→
ACCUMULATION
Iterative Sum
→
RECURRENCE
S(n) = S(n-1) + n²
→
SOURCE CODE
C++ Implementation

Formula Method
Direct Closed Computation

Direct evaluation via algebraic equation.

Time: O(1) Space: O(1)

Iterative Method
Accumulator Loop Loop

Summation loop adding terms sequentially.

Time: O(n) Space: O(1)

Recursive Method
Inductive Recurrence Call

Uses recurrence: $S(n)=S(n-1)+n^2$
Mirrors the inductive proof structure.

Time: O(n) Space: O(n)

Key Connection: **Induction ⇒ Recurrence ⇒ Recursive**

ALGORITHM PARADIGMS

SOFTWARE REALIZATION • C++ CODE ARCHITECTURE

C++ Implementation — Three Approaches

1. Formula Method — O(1)

```
typedef unsigned long long ull; ull sumFormula(ull n) { return n * (n + 1) * (2 * n + 1) / 6; }
```

Pure arithmetic computation.

2. Iterative Method — O(n)

```
ull sumIterative(ull n) { ull sum = 0; for (ull i = 1; i <= n; i++) { sum += i * i; } return sum; }
```

Linear loop accumulator.

3. Recursive Method — O(n)

```
ull sumRecursive(ull n) { if (n == 0) return 0; // Base case return sumRecursive(n - 1) + n * n; }
```

Direct reflection of induction.

Recursion Stack: S(5) → S(4)+5² → S(3)+4² → S(2)+3² → S(1)+2² → S(0)+1² → Return 55

Base ↔ Step

STAGE 4: COMPUTATIONAL SOLVED PROBLEM

Let's Solve a Problem: n = 20

> “What is $1^2 + 2^2 + 3^2 + \ldots + 20^2$?”

Interactive Inquiry

ALGEBRAIC SUBSTITUTION (N = 20)

$$S(20) = 20 \cdot (20 + 1) \cdot (2 \cdot 20 + 1) / 6$$

$$= 20 \times 21 \times 41 / 6$$

$$= 17{,}220 / 6$$

EVALUATED RESULT: **2870**

g++ -O3 ./sum_squares

```
$ ./sum_squares
Enter n: 20
Formula Method : 2870
Iterative Method : 2870
Recursive Method : 2870
```

100% Deterministic Agreement Across All 3 Algorithms

QUESTION (n=20) → ALGEBRAIC CALCULATION → ANSWER (2870) → PROGRAM VERIFICATION ✓

Three Methods — One Mathematical Result

Evaluation Metric	Formula Method	Iterative Method	Recursive Method
Algorithmic Approach	Closed mathematical formula	Definite loop summation	Recursive call stack
Time Complexity	O(1) [Optimal]	O(n) [Linear]	O(n) [Linear]
Auxiliary Space	O(1) [Constant]	O(1) [Constant]	O(n) [Call Stack]
Mathematical Link	Proved closed identity	Direct series definition	Inductive step structure
Best Use Case	Arbitrary large inputs n	Hardware lacking dividers	Teaching proof reflection

“Which method achieves O(1) time complexity?” ➔ Formula Method

3 PATHS → 1 RESULT

PBL Viva Point: Discrete mathematics transforms an O(n) accumulator into an instantaneous O(1) formula.

EMPIRICAL BENCHMARKING • DATA VALIDATION

Experimental Verification

Input (n)	Formula O(1)	Iterative O(n)	Recursive O(n)	Consistency
5	55	55	55	Match ✓
10	385	385	385	Match ✓
20	2,870	2,870	2,870	Match ✓
100	338,350	338,350	338,350	Match ✓
1000	333,833,500	333,833,500	333,833,500	Match ✓

TEST INTERVAL

All n ∈ [0, 2000] tested.

CONVERGENCE

Zero discrepancy observed.

DATA TYPE SAFETY

64-bit unsigned integers.

Report Finding: All three methods agreed for the tested range from n = 0 to 2000.

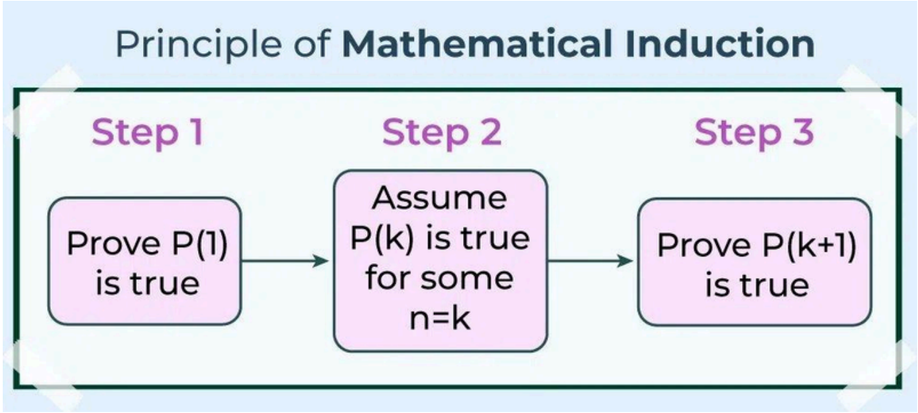
PROJECT SYNTHESIS & REAL-WORLD EXTENSIONS

From Proof to Program

TEAM CONCLUSION

PROBLEM › PATTERN › INDUCTION › PROOF › ALGORITHM › C++ CODE › VERIFICATION

KEY RESEARCH FINDINGS



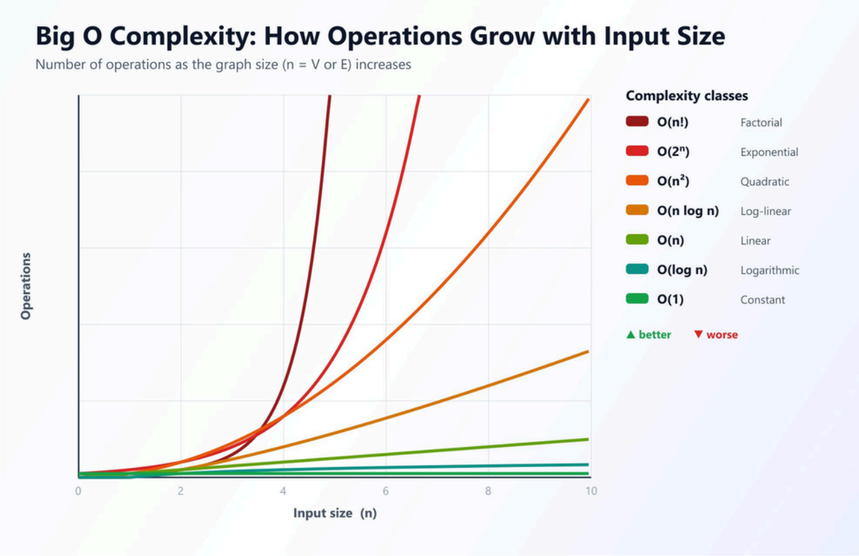
Universal Proof: True for all positive integers.

Isomorphism: Recursion directly mirrors inductive reasoning.

Optimization: Closed formula cuts O(n) loops to O(1).

PBL Synthesis: Connected discrete mathematics with programming.

COMPUTATIONAL APPLICATIONS



- Number Theory & Series
- Algorithm Analysis (Big-O)
- Variance & Statistics
- Physics & Inertia Calculations
- Loop Invariant Program Verification

THANK YOU — Open for Viva Questions & Discussion